Regression and the Single Neuron

From the Chain Rule to ChatGPT: The Mathematics Behind AI

John Mitchell · Clark College · WAMATYC 2026

May 2026

The Learning Problem

A machine learning model is a parameterized function. Given an input x, it produces a prediction $\hat{y}$ by applying a function f whose behavior is controlled by parameters θ:

$$\hat{y} = f(x;\, \theta)$$

Learning means adjusting θ so that predictions match observed outputs y as closely as possible. The entire field of supervised machine learning — from linear regression to GPT-3 — is a set of answers to three questions:

- What is the form of f?
- How do we measure the gap between $\hat{y}$ and y?
- How do we adjust θ to close that gap?

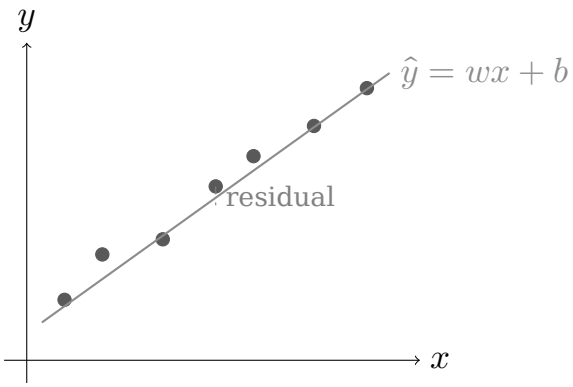
Every section of this handout series answers these questions at increasing scale. We start with two parameters. GPT-3 has 175 billion. The answers — and the underlying mathematics — are structurally the same throughout.

Linear Regression

The simplest parameterized function is a line. For a single input feature x, the model is:

$$\hat{y} = wx + b$$

The parameters are the slope w (weight) and intercept b (bias). Together, $\theta = (w, b)$ — two numbers that define the line.



The Loss Function: Mean Squared Error

To measure how well the line fits, we compute the average squared residual over all n training examples:

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2 = \frac{1}{n} \sum_{i=1}^{n} (wx_i + b - y_i)^2$$

Squaring serves two purposes: it makes all residuals positive, and it penalizes large errors more heavily than small ones. The loss L is a function of the parameters — it measures how wrong our current θ is.

> **Historical note.** Least squares regression was developed independently by Adrien-Marie Legendre (1805) and Carl Friedrich Gauss (around 1795, published 1809). Gauss used it to predict the orbit of the asteroid Ceres after it was lost behind the sun — successfully locating it when telescopes returned to the right patch of sky. The method predates modern statistics by a century and remains the foundation of supervised learning today.

Gradient Descent

The loss surface $L(w, b)$ is a bowl-shaped paraboloid in (w, b) space. The minimum of the bowl is the best-fit line. We find it by repeatedly stepping downhill.

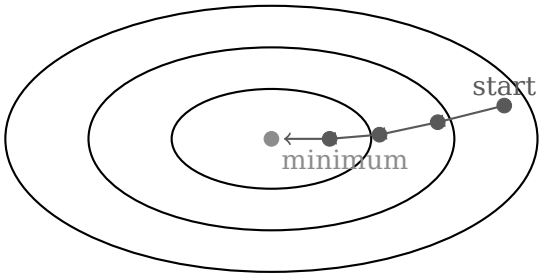
The gradient $\nabla_\theta L$ points in the direction of steepest *increase*. To minimize loss, we step in the opposite direction:

$$\theta \leftarrow \theta - \alpha \nabla_\theta L(\theta)$$

Written out for each parameter:

$$w \leftarrow w - \alpha \frac{\partial L}{\partial w} \qquad b \leftarrow b - \alpha \frac{\partial L}{\partial b}$$

The learning rate α controls step size. Too large and the updates overshoot the minimum; too small and convergence is slow.



Contour plot of $L(w, b)$

Going deeper: the partial derivatives of MSE. Expanding $L = \frac{1}{n}\sum(wx_i + b - y_i)^2$ and differentiating by the chain rule:

$$\frac{\partial L}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}(wx_i + b - y_i)x_i \qquad \frac{\partial L}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}(wx_i + b - y_i)$$

Each partial is an average of (residual $\times$ input). The chain rule is already present here: differentiating $(wx_i + b - y_i)^2$ requires the derivative of a composition. This is a preview of backpropagation at the simplest possible scale.

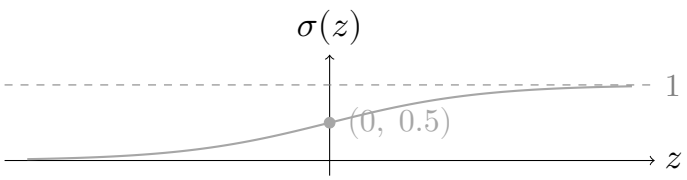
Key result. Gradient descent on MSE loss finds the parameters (w, b) that minimize prediction error. The update rule $\theta \leftarrow \theta - \alpha\,\nabla L$ is the same algorithm we will apply to networks with billions of parameters.

From Regression to Classification

Linear regression predicts a continuous value. For classification — is this email spam? is this tumor malignant? — we need an output in $[0, 1]$ interpretable as a probability.

The fix is to pass the linear combination through an *activation function*. The logistic sigmoid is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



The full logistic regression model is:

$$\hat{y} = \sigma(wx + b) = \frac{1}{1 + e^{-(wx+b)}}$$

Why Not MSE for Classification?

MSE combined with a sigmoid output produces a non-convex loss surface with many local minima. The appropriate loss for binary classification is *cross-entropy*:

$$L = -\frac{1}{n}\sum_{i=1}^{n}\Big[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\Big]$$

Going deeper: cross-entropy from maximum likelihood. If we model each label y_i as a Bernoulli random variable with success probability $\hat{y}_i$, the log-likelihood of the training data is:

$$\ell(\theta) = \sum_{i=1}^{n}\big[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)\big]$$

Minimizing cross-entropy loss is identical to maximizing this log-likelihood. The loss function is the negative log-likelihood of the data under the model — a direct bridge to maximum likelihood estimation in statistics.

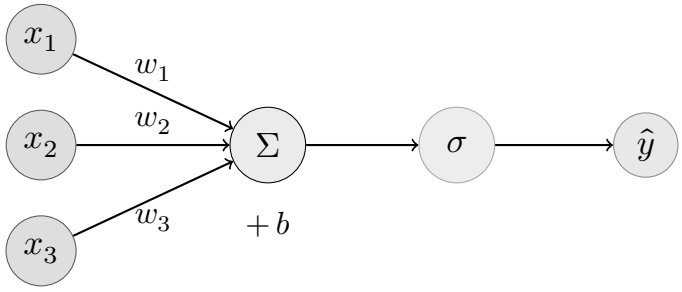
Common misconception. Cross-entropy is not a geometric distance. It measures the difference between two probability distributions — the true labels and the model's predictions. It comes from information theory, where it quantifies the cost of encoding one distribution using a code optimized for another. The fact that it also arises from maximum likelihood estimation is not a coincidence; both frameworks are measuring the same thing from different starting points.

Historical note. Logistic regression was formalized by statistician David Cox in 1958, though the sigmoid curve appeared earlier in models of population growth (Verhulst, 1845). Cross-entropy as a loss function entered machine learning through Claude Shannon's information theory (1948). The pairing of sigmoid activation with cross-entropy loss — standard in neural networks today — is optimal in a precise information-theoretic sense: the cross-entropy gradient with respect to the pre-activation is simply $\hat{y} - y$, which makes the update rule exceptionally clean.

The Single Neuron

A single artificial neuron takes one or more inputs, computes a weighted sum, adds a bias, and passes the result through an activation function:

$$\hat{y} = \sigma\left(\sum_{j=1}^{d} w_j\, x_j + b\right)$$



The neuron is a generalization of logistic regression to multiple inputs. With a single input and the sigmoid activation, it *is* logistic regression. With d inputs, the parameters are the weights $w_1, \ldots, w_d$ and bias b — a total of $d + 1$ numbers.

Key result. A single neuron is already a neural network. The diagram above — inputs, weighted sum, activation, output — is the architectural unit that scales without structural change to GPT-3. We currently have $d + 1$ parameters. The architecture is complete.

Historical note. The artificial neuron was introduced by Warren McCulloch and Walter Pitts in 1943 as a mathematical abstraction of biological neurons. Frank Rosenblatt's Perceptron (1958) added the learning rule — adjusting weights based on prediction errors — and was the first trainable single-layer model. Marvin Minsky and Seymour Papert's *Perceptrons* (1969) proved that a single neuron cannot learn the XOR function, temporarily dampening enthusiasm for the field. The resolution — stacking neurons in layers — is the subject of Handout 2.

Summary: Parameter Count and the Through-Line

Model	Parameters	Loss
Linear regression (1 feature)	2	MSE
Logistic regression (1 feature)	2	Cross-entropy
Single neuron (d inputs)	$d + 1$	Cross-entropy

The learning algorithm in every row is identical: gradient descent on a differentiable loss function. What changes as we move to deeper networks is not the update rule but how we compute $\nabla_\theta L$ efficiently across many layers. That computation is backpropagation — and backpropagation is the chain rule. Handout 2 builds the network; Handout 2 closes with the payoff.

The through-line. Every model in this series learns by the same mechanism: compute a loss, compute its gradient with respect to all parameters, take a small step downhill. We have established this mechanism at the smallest possible scale. The chain rule is what makes it tractable at any scale. That is the subject of Handout 2.